

Assignment: Project PEAS Assessment

CS 4300 - Project PEAS Assessments - 2D Maze Solver

When it comes to a maze, you are presented with a simple task: get from point A to point B. This in of itself may not be very useful, though, that is dependent on what point A and B are. A useful AI agent would help plan the most optimal and efficient path in order to get from said point A and B in order to get to the end of the maze in the least possible amount of time given a variety of different mazes ranging from simple to complex in a two-dimensional environment.

The Performance Measure:

The negative of the following elements will be summed together. The best agent will have the greatest performance measure. A perfect score of 0 would imply the agent reached the goal without any inefficiencies (which is highly unlikely).

5 (Total number of moves): The number of moves the agent makes, including both correct moves and backtracking. The fewer moves, the more efficient the agent is.

10 (Number of dead-ends encountered): This measures how often the agent gets stuck in dead-ends. Dead-ends could indicate suboptimal pathfinding or poor decision-making.

20 (Number of unnecessary loops/cycles): This metric penalizes the agent for revisiting the same part of the maze unnecessarily (i.e., going in circles). A higher number of loops would suggest inefficient exploration strategies.

The performance measure would look something like this:

$$\text{Performance Measure} = -(5 * (\text{Total Moves}) + 10 * (\text{Dead-Ends}) + 20 * (\text{Loops}))$$

The Environment is:

- Observability: **Partially Observable**: Each maze generation results in an entirely new and unknown layout for the agent. The agent does not have access to the full maze layout from the start. It can only observe its immediate surroundings (like the adjacent cells or a small portion of the maze), which means it must explore and map out the maze incrementally.
- Uncertainty: **Deterministic**. Most actions (e.g., moving in a direction) have predictable outcomes. The agent moves one step in the chosen direction if no walls or obstacles block the path. There's no randomness in the outcomes of its actions, seeing as how there are no unexpected elements in the maze (e.g., traps).

- Duration: **Sequential**. The agent's actions have a sequential nature, where it has to plan step by step. Each move brings it closer or farther from the goal, and decisions build upon the current state, requiring ongoing recalculation as the agent explores more of the maze.
- Stability: **Static**. Once the maze is generated, its layout (walls, passages, dead-ends) doesn't change during the agent's interaction with it. The agent can explore and learn the structure, but the maze itself remains fixed throughout a single run.
- Granularity: **Discrete**. The agent's position is represented by discrete grid cells. Moves occur in fixed units (e.g., up, down, left, right) and time is measured in turns or steps. Every action changes the agent's position by a discrete amount.
- Participants: **Single-Agent**. The agent is the only entity interacting with the environment, focused on solving the maze alone.
- Knowledge: **Mostly Unknown**: At the beginning of each new maze, the agent's knowledge is reset to unknown, and it has to explore and discover the maze anew. Though the agent still knows the general rules of movement and understands the goal (reach point B from point A).

The Actions (Actuators):

- **Move Up**: The agent moves one cell upward (north) in the maze if no wall or obstacle is blocking the way.
- **Move Down**: The agent moves one cell downward (south) in the maze if possible.
- **Move Left**: The agent moves one cell to the left (west) in the maze if there is no barrier.
- **Move Right**: The agent moves one cell to the right (east) if the path is clear.

The Percepts (Sensors):

- **Current Position**: Knows its current position in the maze, represented as coordinates (x, y) on the 2D grid, for tracking movement and planning future actions.
- **Adjacent Cells**: The agent perceives the state of the cells immediately adjacent to its current position (up, down, left, right). This allows it to know if there are walls, open paths, or obstacles in those directions.
 - **Wall Present (Yes/No)**: For each adjacent cell, the agent can perceive whether it is blocked by a wall or not.
- **Goal Location**: Knowing where the goal is in the given maze via coordinates (point B).
- **Distance from Goal**: Utilizing either the Euclidean or Manhattan distance to perceive the distance from the goal would help guide its pathfinding decisions.
- **Visited Cells**: Keeping track of already visited cells would help avoid loops or dead ends.
- **Maze Boundaries**: If known, this'll allow the agent to better understand its limits for exploration.

Intended Search Strategy: **A* Search**